Chapter 10

Figure 10.1 — Snake Game — This chapter overview shows the transformation from basic snake movement to complete gameplay by adding random food placement, snake growth mechanics, score tracking, and collision detection between the snake and food objects.

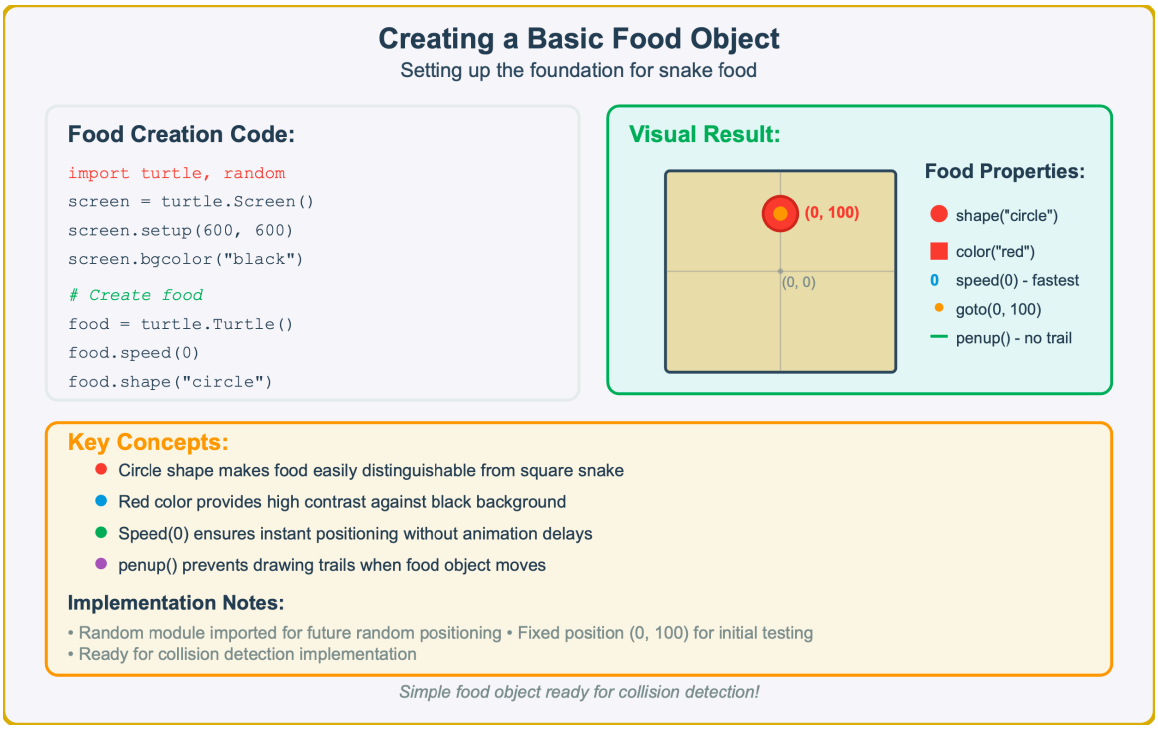

Figure 10.2 — Creating a Basic Food Object. This implementation guide shows how to create the foundation food object for the Snake game, including the essential code setup, visual properties (circle shape, red color), positioning system, and key programming concepts that make the food distinguishable and ready for collision detection.

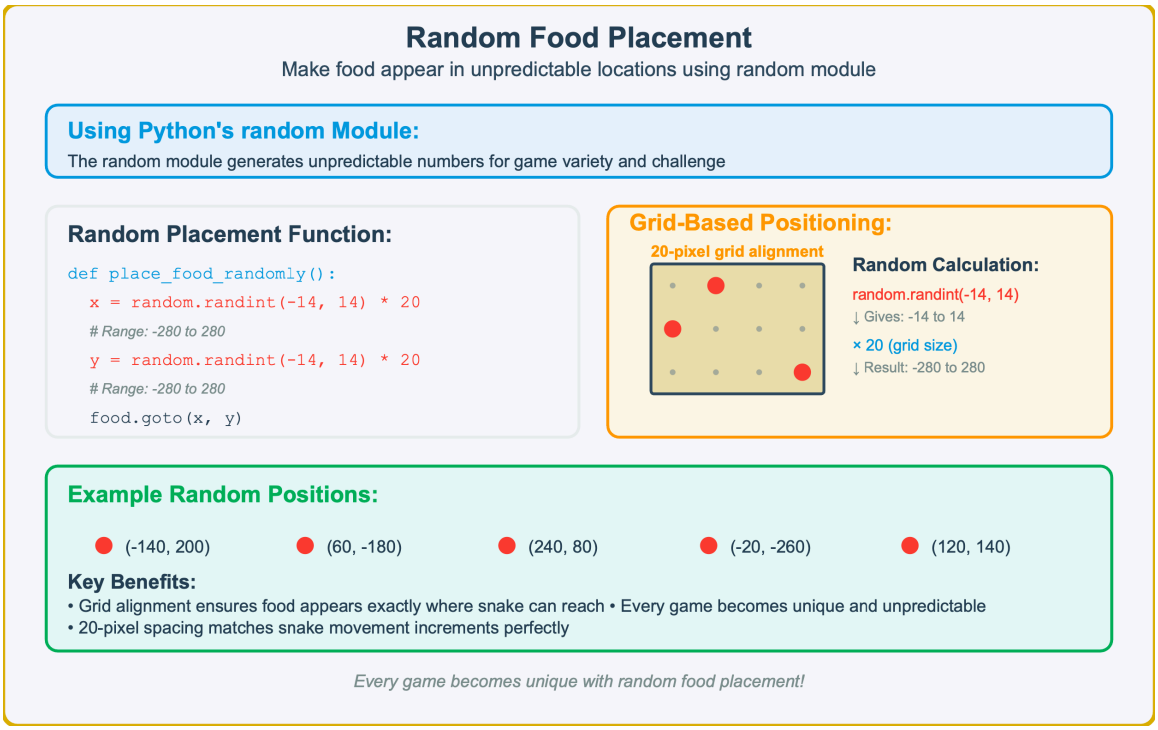

Figure 10.3 — Random Food Placement. This implementation guide demonstrates how to use Python's random module to create unpredictable food positioning in the Snake game, showing the grid—based calculation system that ensures food appears at reachable 20—pixel intervals within the game boundaries.

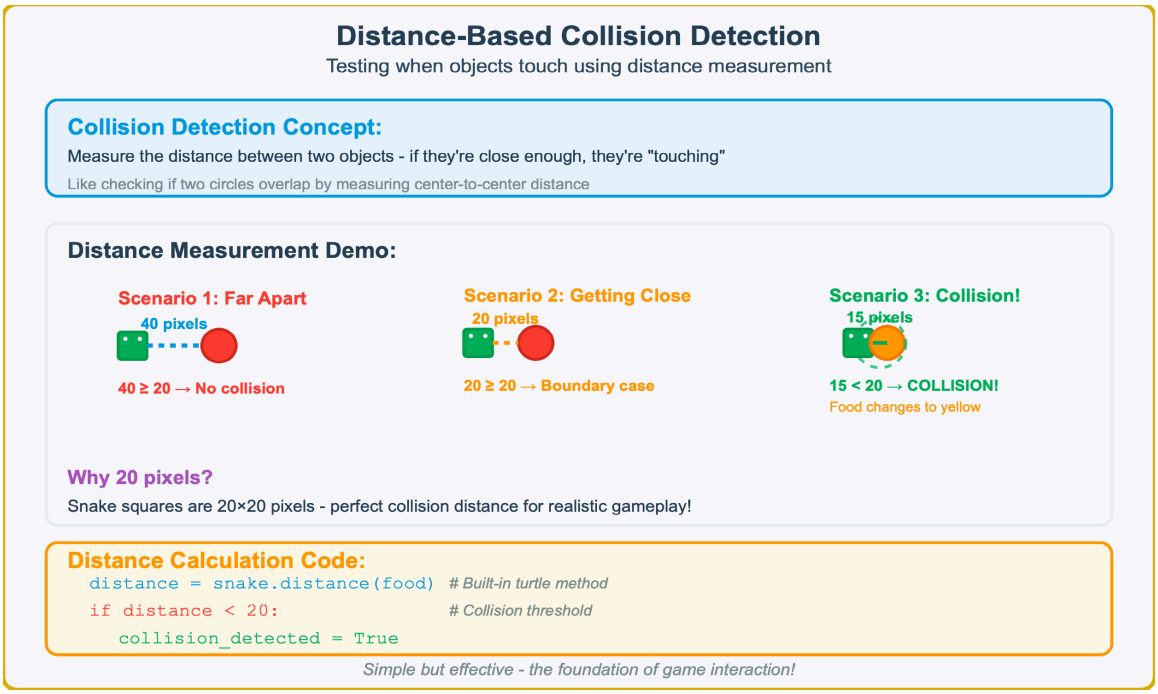

Figure 10.4 — Distance—Based Collision Detection. This concept demonstration shows how collision detection works by measuring the distance between snake and food objects, illustrating three scenarios: far apart (no collision), boundary case (20 pixels), and collision (less than 20 pixels), along with the essential code implementation.

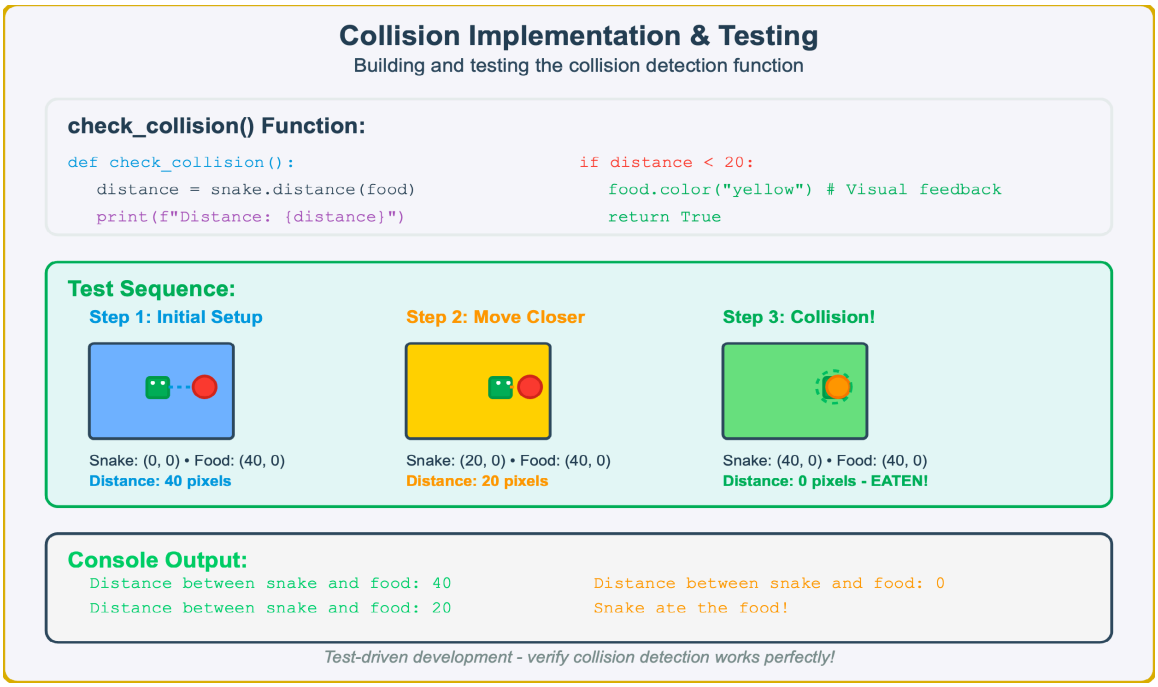

Figure 10.5 — Collision Implementation & Testing. This testing demonstration shows the complete check_collision() function implementation and a three—step test sequence that verifies collision detection: initial setup (40 pixels apart), moving closer (20 pixels), and successful collision (0 pixels) with visual and console feedback.

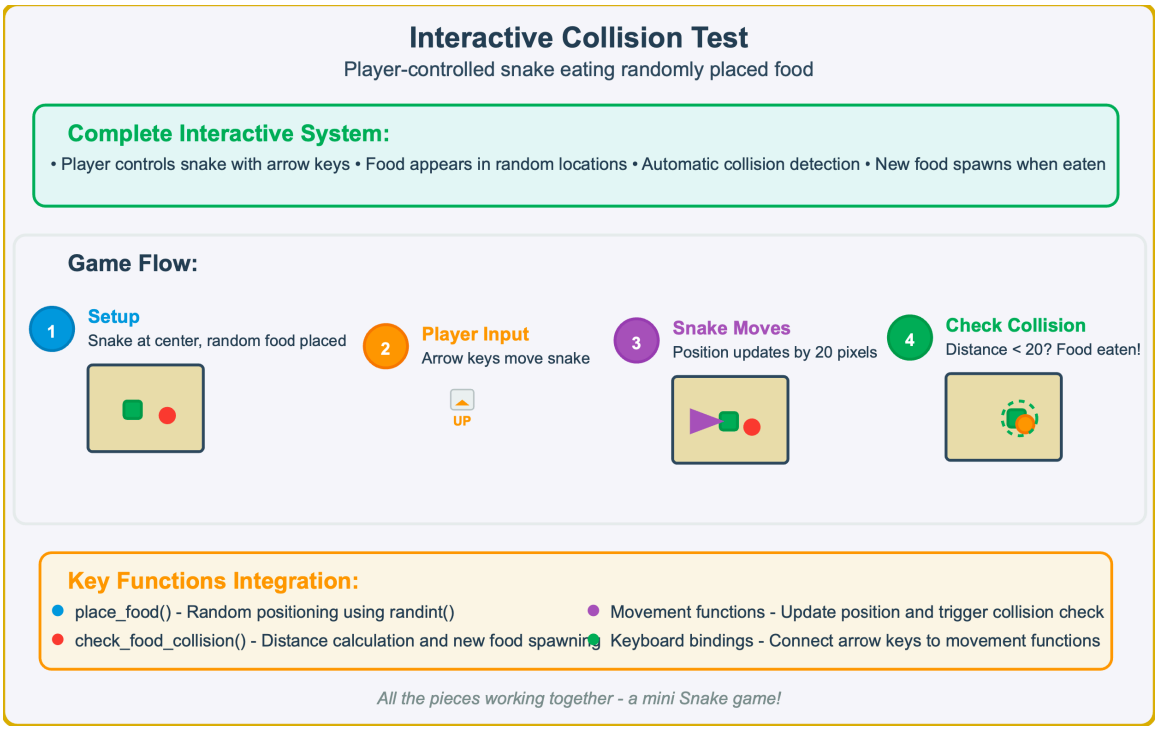

Figure 10.6 — Interactive Collision Test. This system overview demonstrates the complete interactive game flow from setup through player input, snake movement, and collision detection, showing how all the components (random food placement, player controls, movement functions, and collision detection) work together to create a functional mini Snake game.

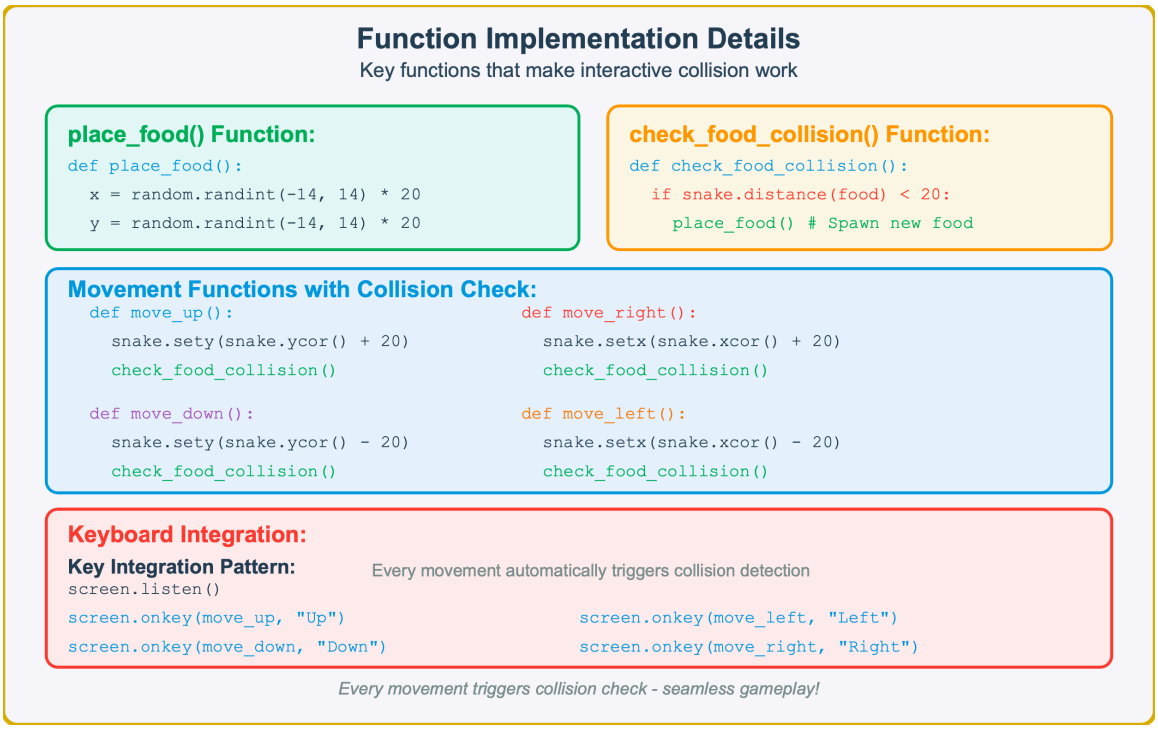

Figure 10.7 — Function Implementation Details. This implementation reference shows the key functions that enable interactive collision gameplay: place_food() for random positioning, check_food_collision() for detection and respawning, movement functions that integrate collision checking, and keyboard bindings that connect player input to game actions.

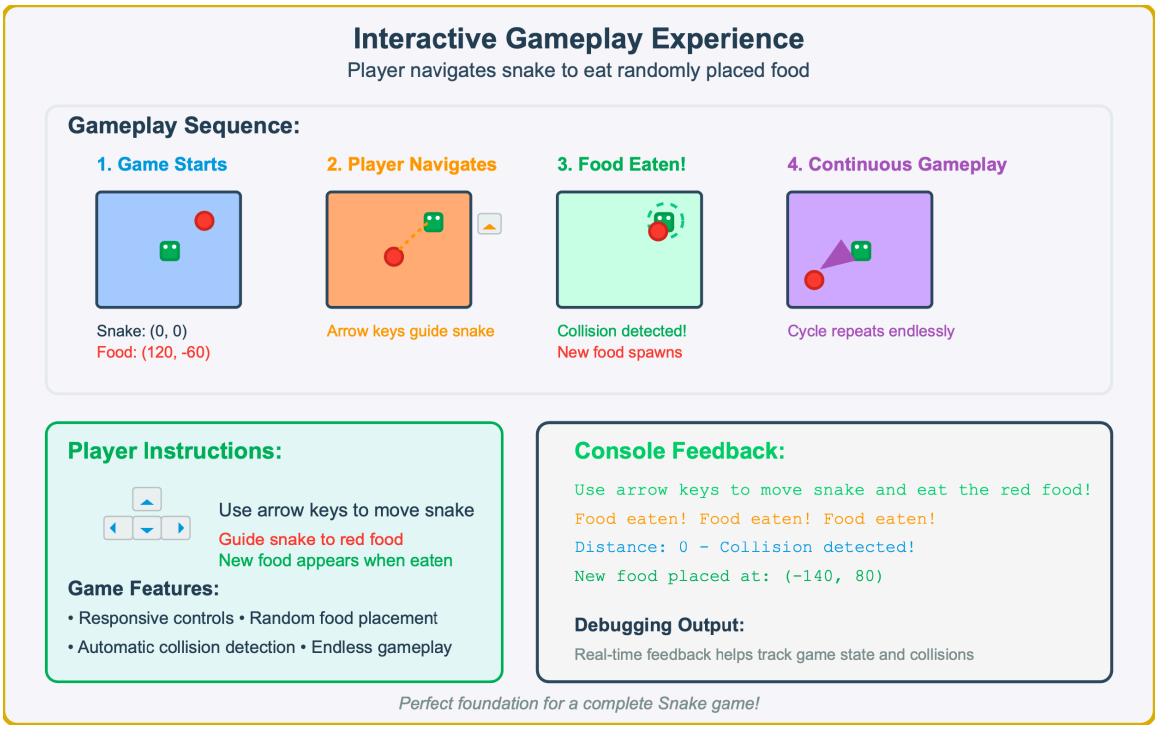

Figure 10.8 — Interactive Gameplay Experience. This comprehensive gameplay demonstration shows the complete player experience: initial game setup with random food placement, player navigation using arrow keys, collision detection and food consumption, continuous gameplay cycle, along with player instructions and console feedback for debugging.

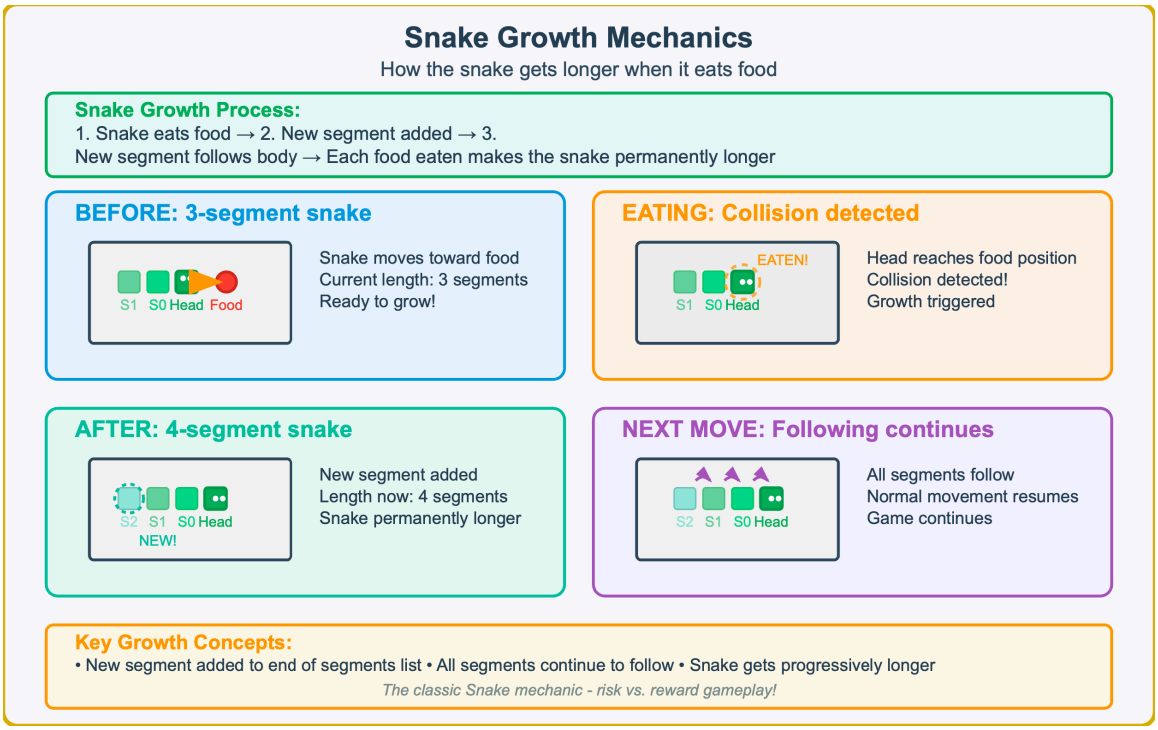

Figure 10.9 — Snake Growth Mechanics — Adding Segments Through Food Consumption. This sequence demonstrates how eating food dynamically adds new segments to the snake's body, creating the core challenge progression in Snake games.

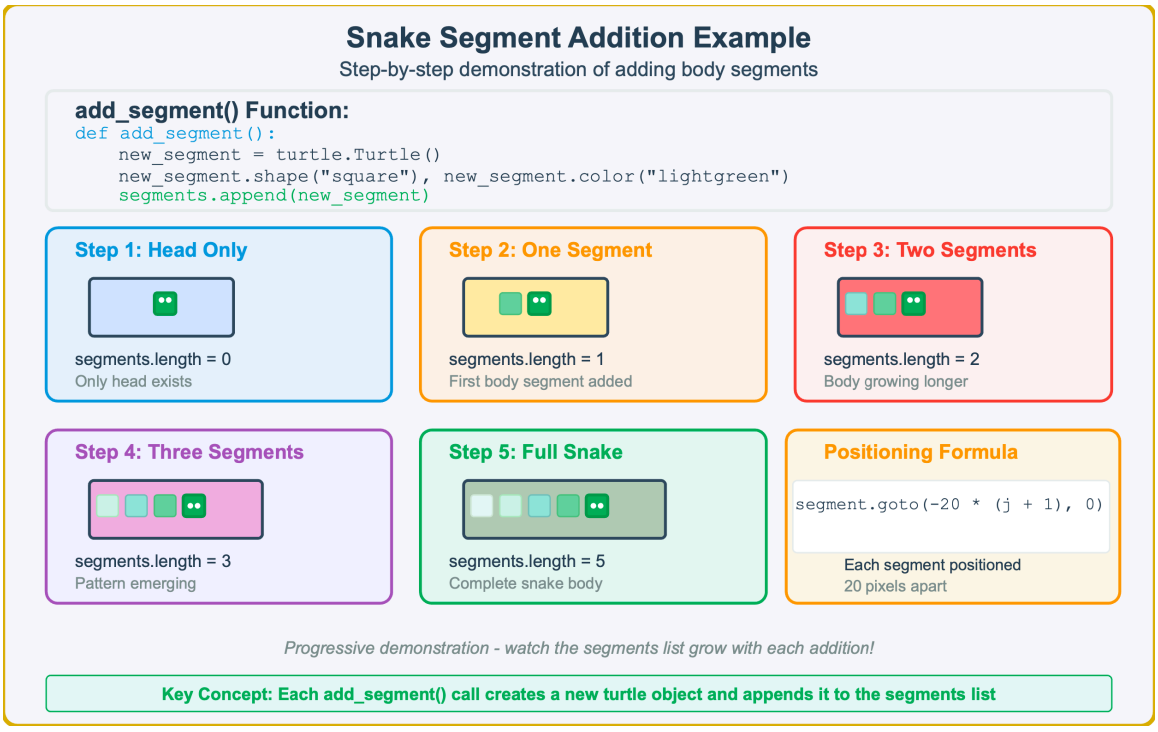

Figure 10.10 — Progressive Snake Segment Addition — Building the Body List. This demonstration shows how each call to add_segment() creates a new turtle object and appends it to the segments list, progressively building the snake's body from head—only to a full 5—segment snake.

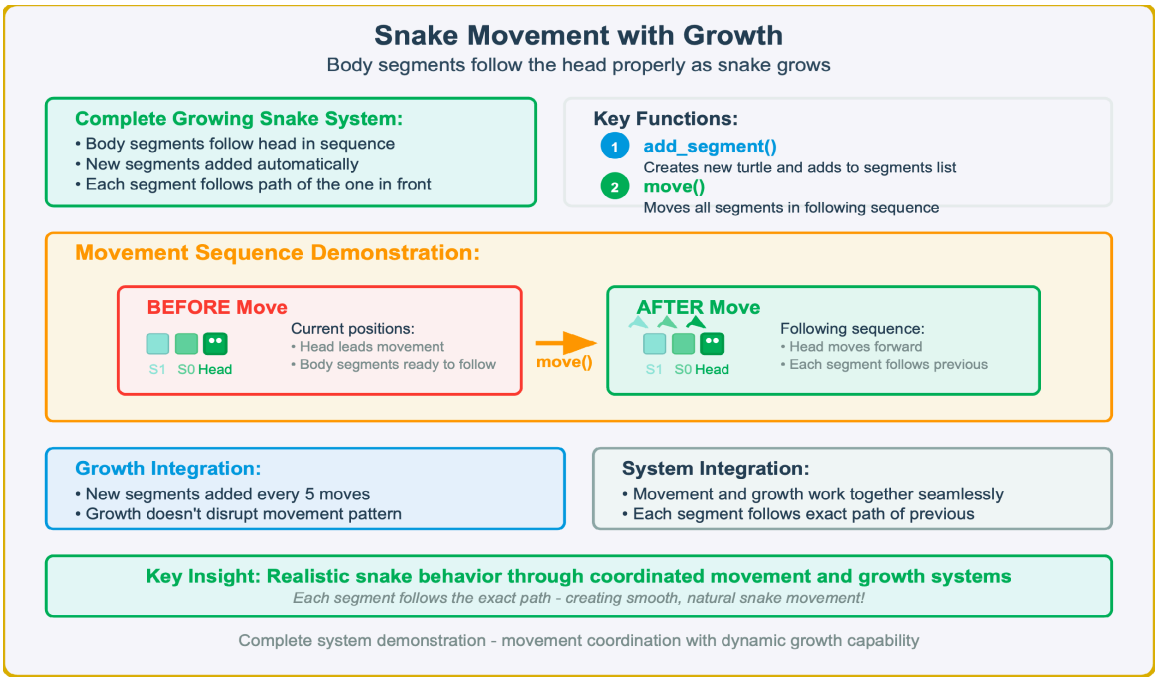

Figure 10.11 — Integrated Snake Movement and Growth System. This overview demonstrates how the add_segment() and move() functions work together to create realistic snake behavior where body segments follow the head in sequence, with new segments automatically integrating into the following pattern.

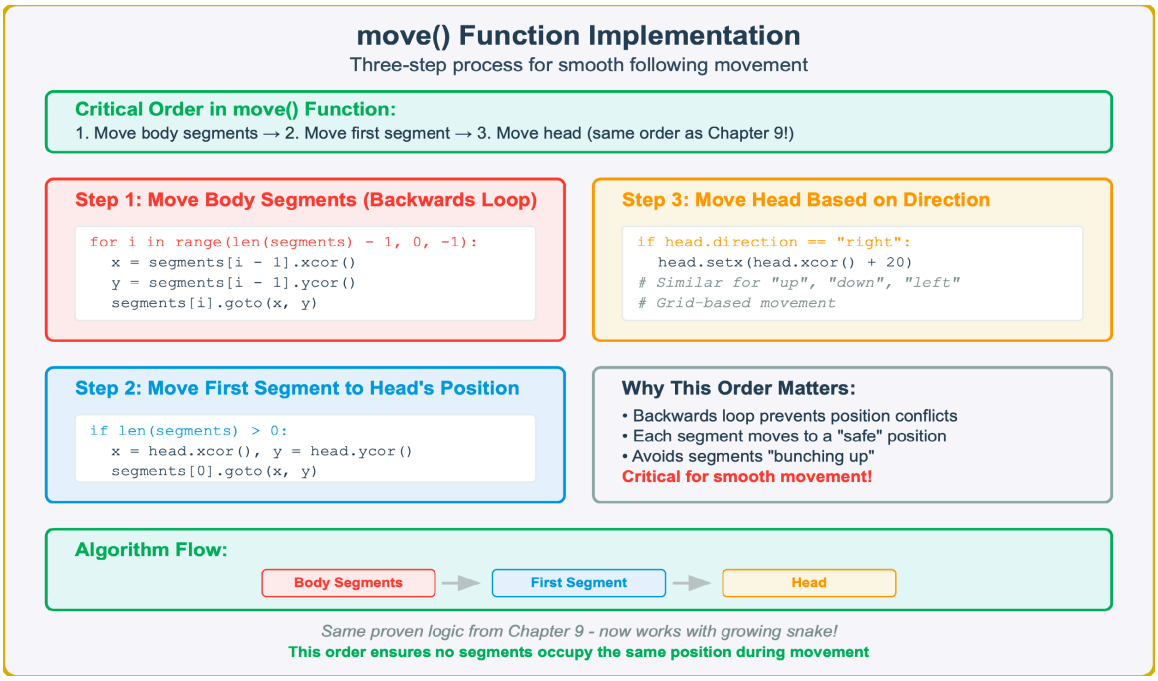

Figure 10.12 — move() Function Three—Step Implementation. The critical order prevents segments from “bunching up”: (1) move body segments backwards through the list, (2) move first segment to head’s current position, (3) move head based on direction. This proven logic from Chapter 9 now works with the growing snake.

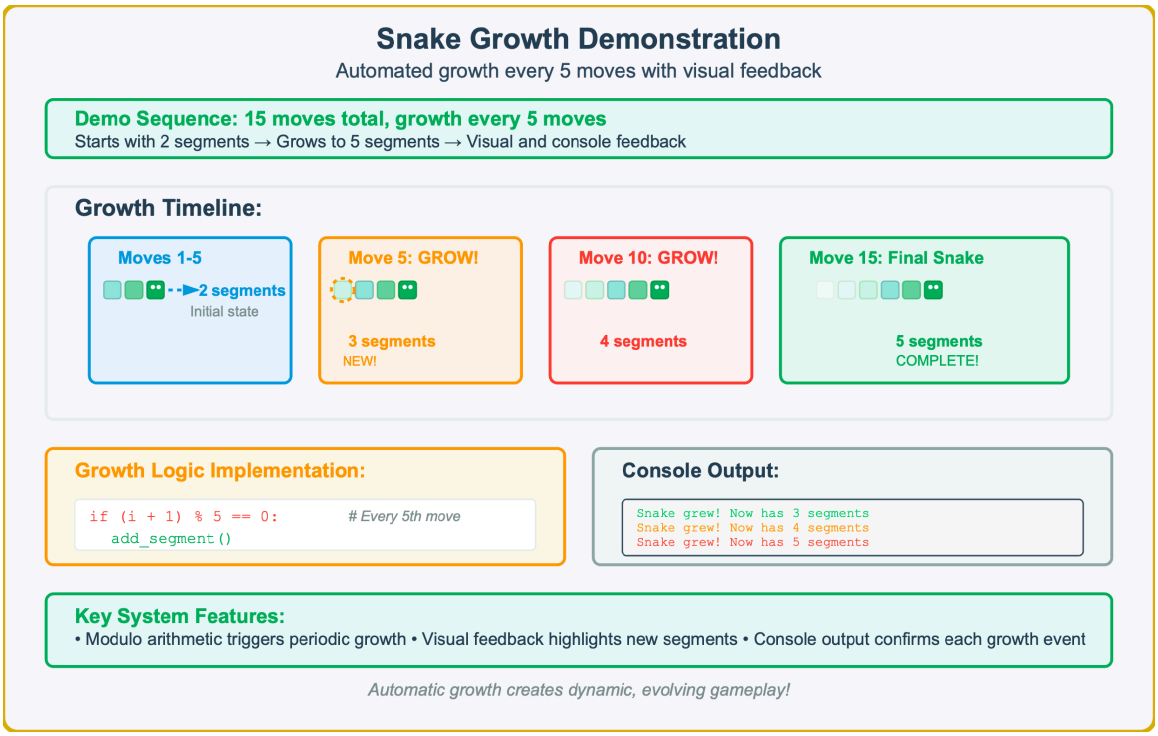

Figure 10.13 — Automated Snake Growth Timeline — 15—Move Demonstration. This sequence shows automated growth every 5 moves, starting with 2 segments and growing to 5 segments. The growth logic uses modulo arithmetic to trigger expansion, with visual feedback and console output confirming each growth event.

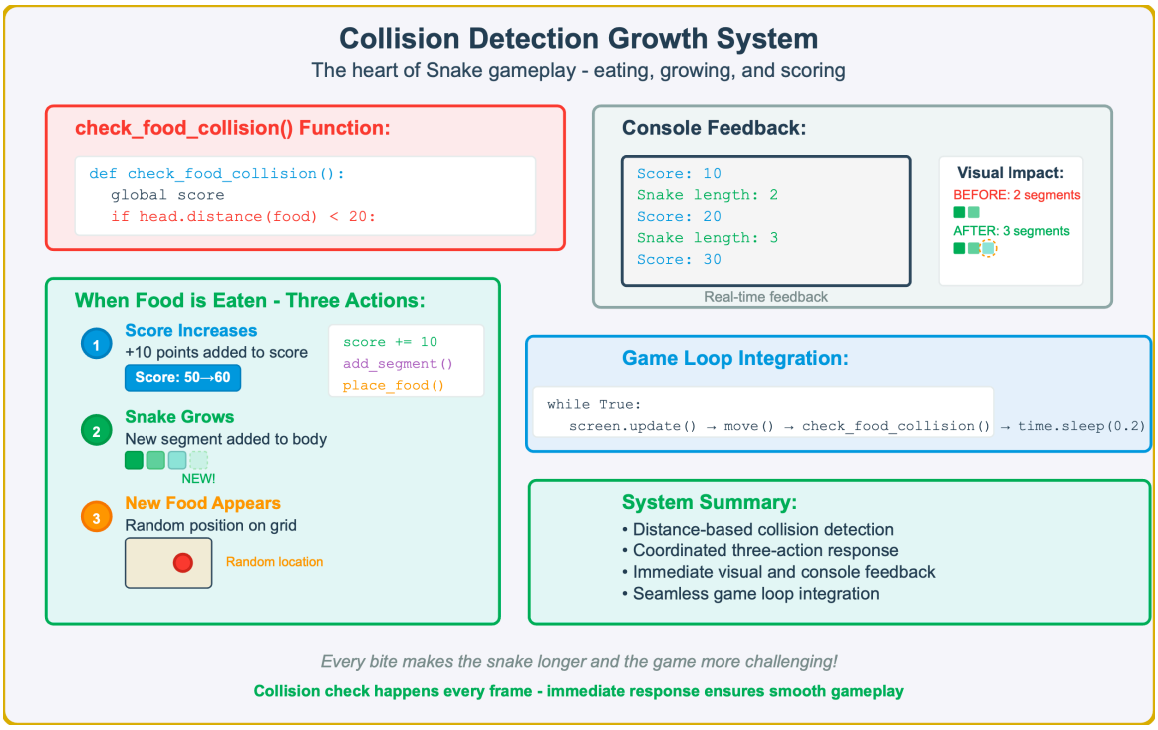

Figure 10.14 — Food Collision and Growth System. The check_food_collision() function detects when the snake head reaches food, triggering three coordinated actions: score increases (+10 points), snake grows (add_segment()), and new food appears (place_food()). This system integrates into the main game loop for immediate response.

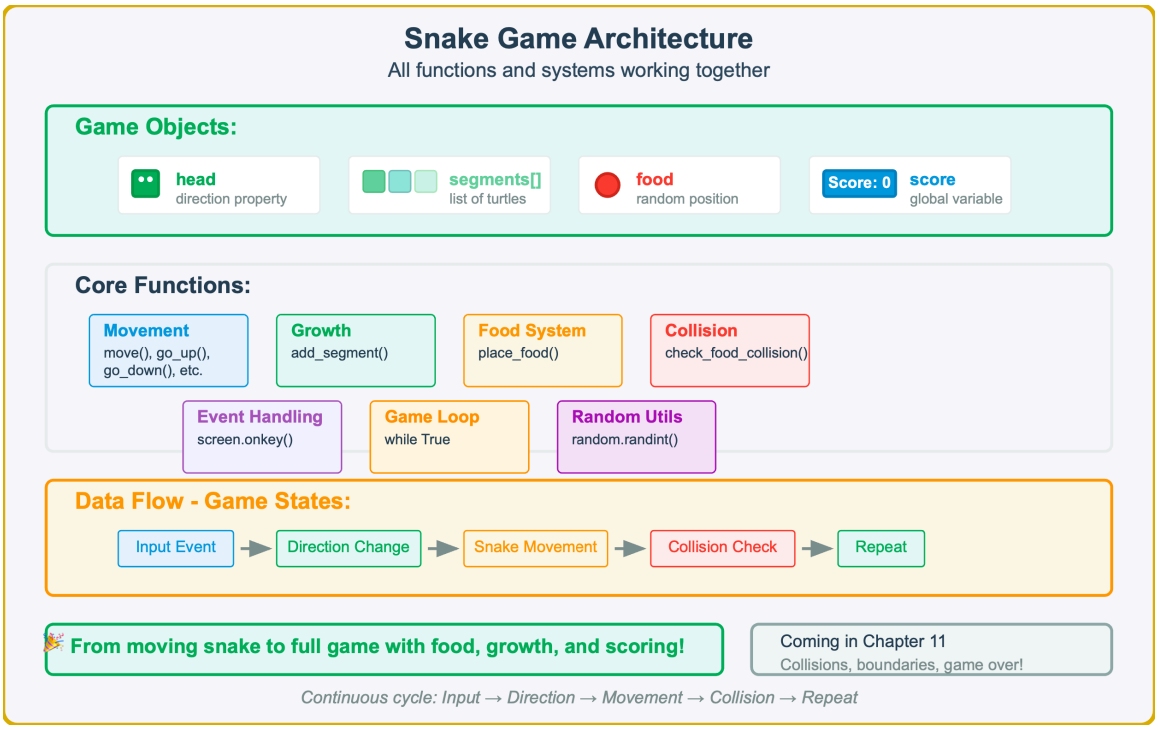

Figure 10.15 — Snake Game Architecture Overview. This comprehensive view shows all game objects (head, segments, food, score), core functions organized by category (movement, growth, food system, collision), and the continuous data flow cycle from input events through direction changes, movement, and collision detection.

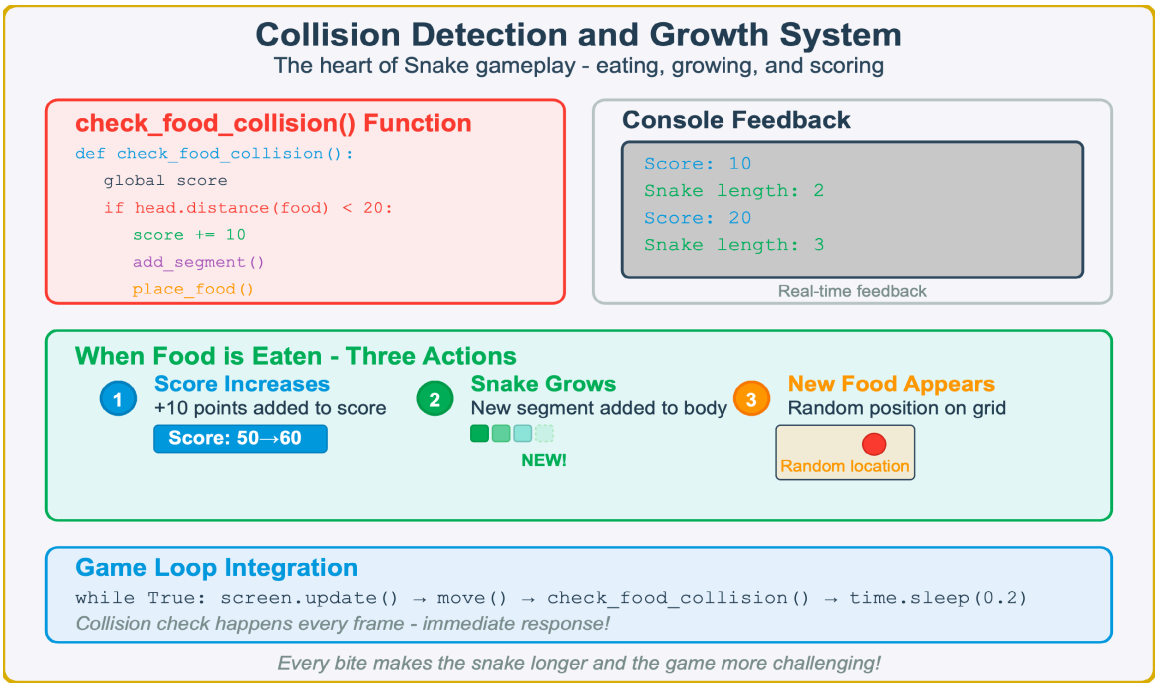

Figure 10.16 — Collision Detection and Growth System. The core mechanics of Snake gameplay showing how the check_food_collision() function manages eating, growing, and scoring. When the snake's head gets within 20 pixels of food, three sequential actions occur: score increases by 10 points, a new segment is added to the snake's body, and new food appears at a random location. The system integrates seamlessly into the game loop, providing immediate feedback and continuous gameplay progression.

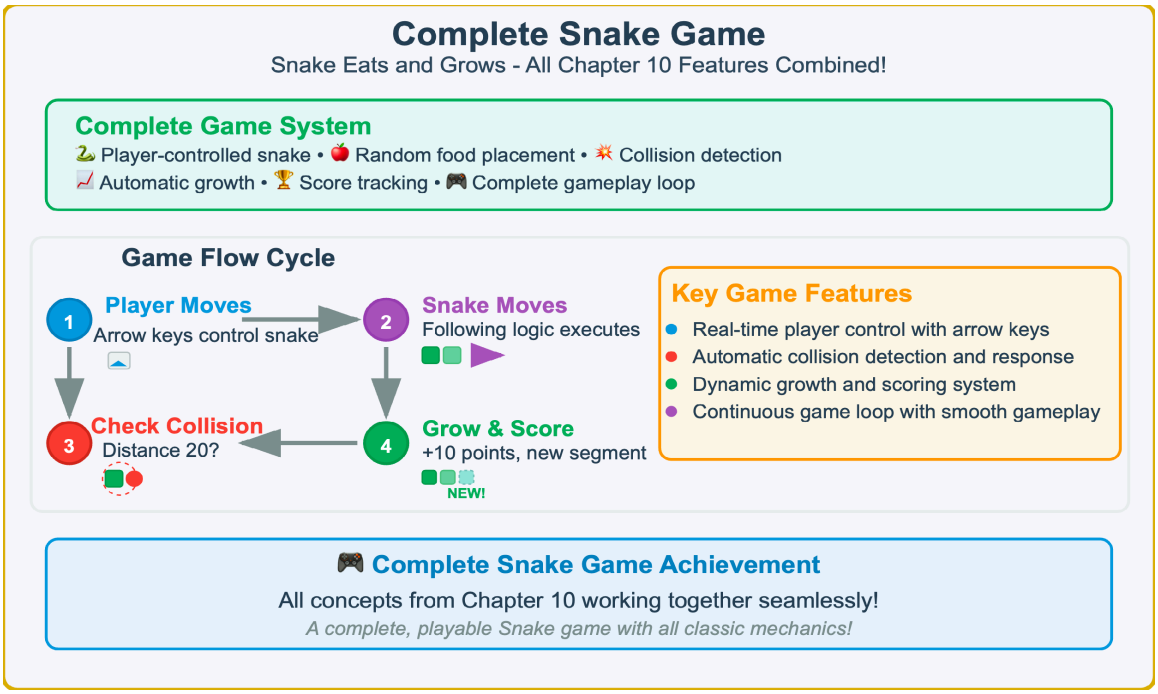

Figure 10.17 — Complete Snake Game Overview. The culmination of Chapter 10's game development concepts, showing how all components work together in a complete Snake game. The central game flow cycle demonstrates the four key steps: player input, snake movement, collision detection, and growth/scoring. The diagram illustrates how player—controlled snake movement, food collision detection, automatic growth, and score tracking combine to create a fully functional game with continuous gameplay.

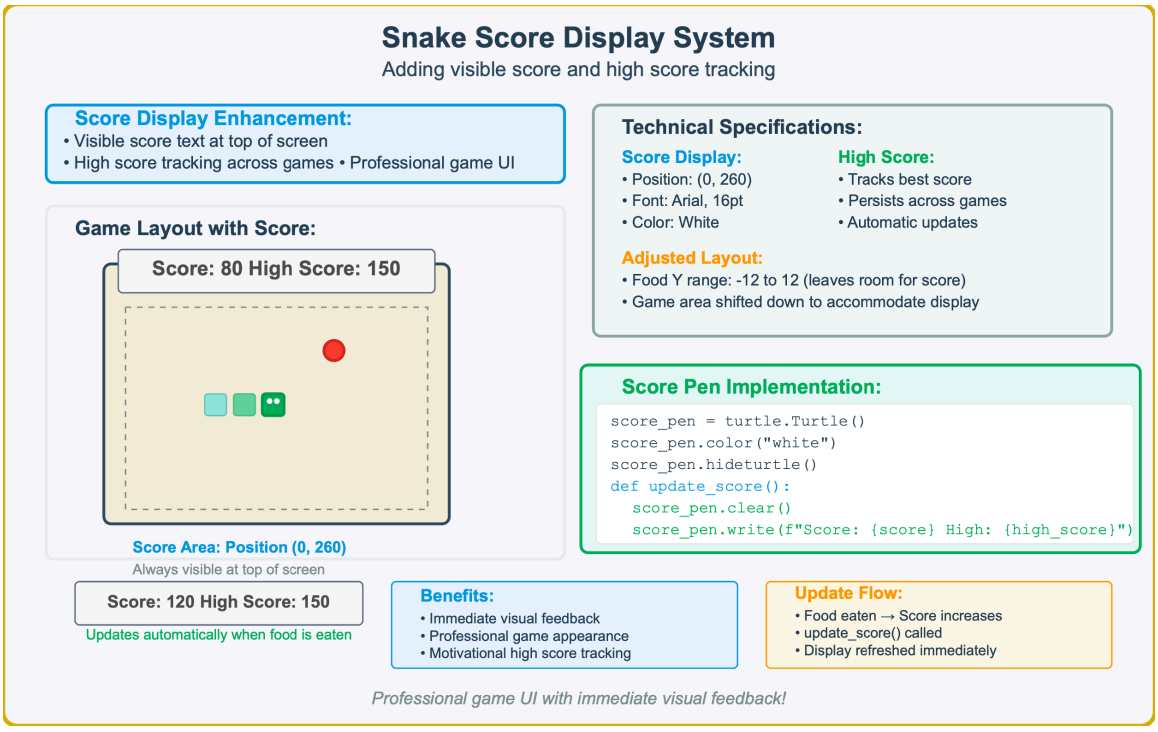

Figure 10.18 — Snake Score Display System — Professional Game UI. This enhancement adds visible score text at the top of the screen (position 0, 260) with high score tracking that persists across games. The score pen implementation uses turtle graphics to display real—time feedback, creating a professional game interface with adjusted layout to accommodate the score display.

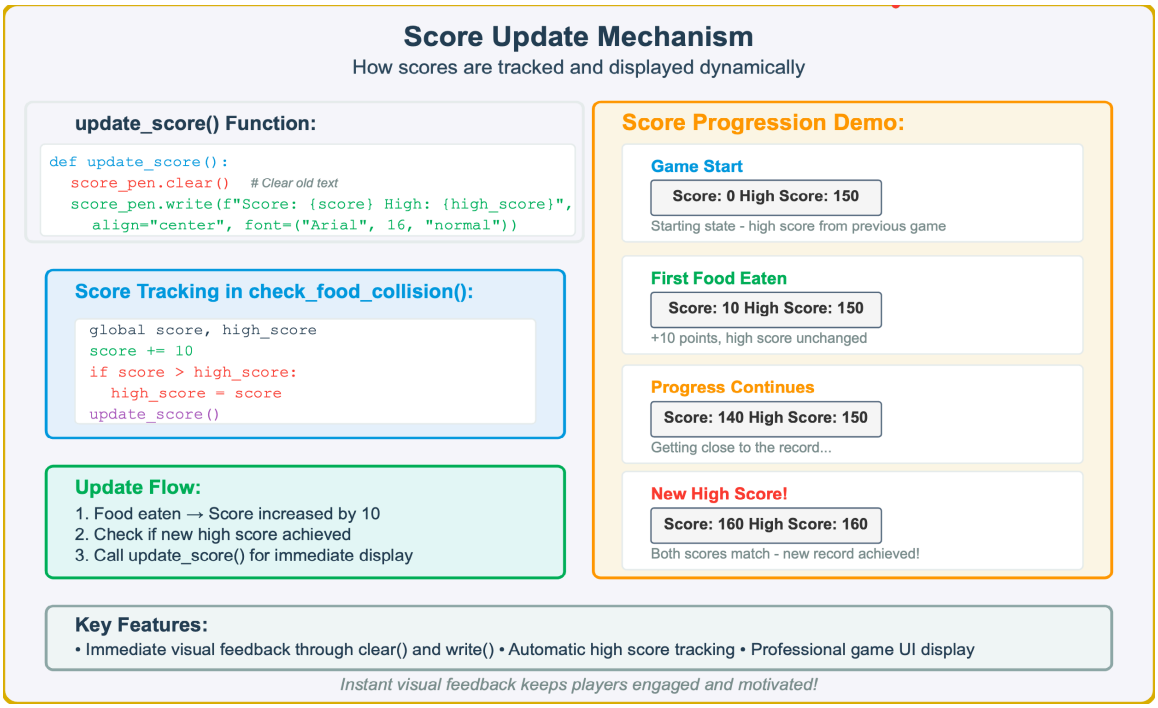

Figure 10.19 — Dynamic Score Update Mechanism. The update_score() function uses score_pen.clear() and score_pen.write() to refresh the display. Score tracking occurs in check_food_collision() where the score increases by 10 points, high score is checked and updated if exceeded, and update_score() is called for immediate visual feedback.

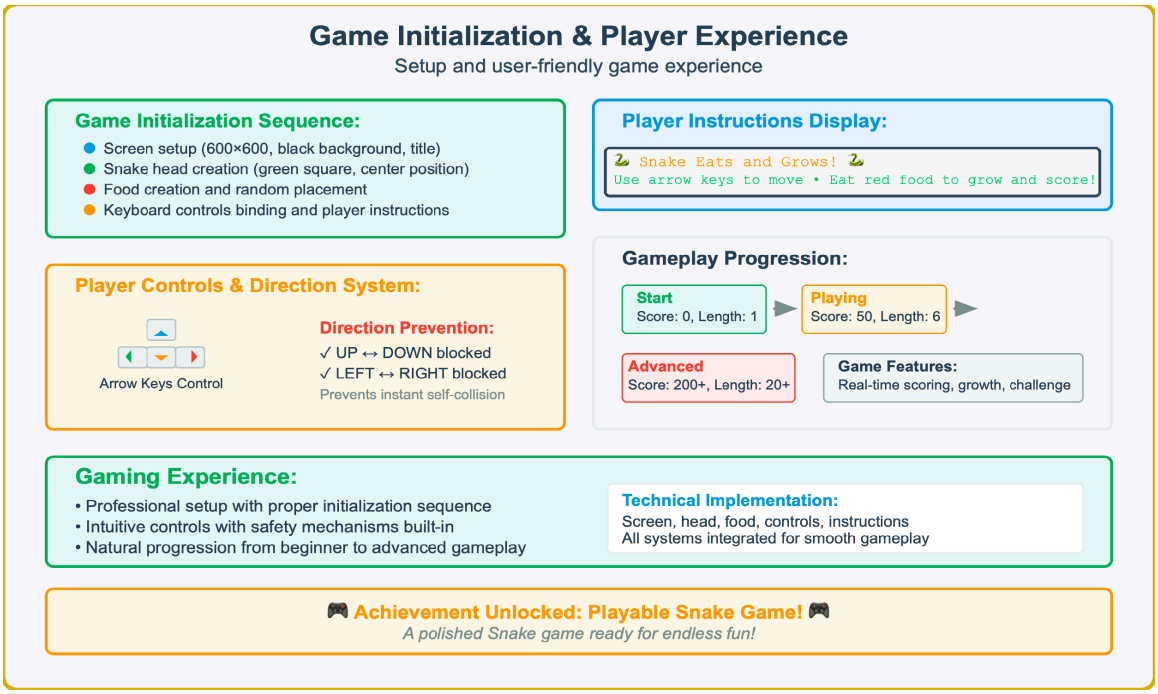

Figure 10.20 — Game Initialization and Player Experience. The game setup includes screen configuration (600×600 black background), snake head creation at center, food placement, and keyboard controls binding. Player controls use arrow keys with direction prevention (UP↔DOWN, LEFT↔RIGHT blocked) to prevent instant self—collision, progressing from starting state (Score: 0, Length: 1) through playing (Score: 50, Length: 6) to advanced gameplay (Score: 200+, Length: 20+).

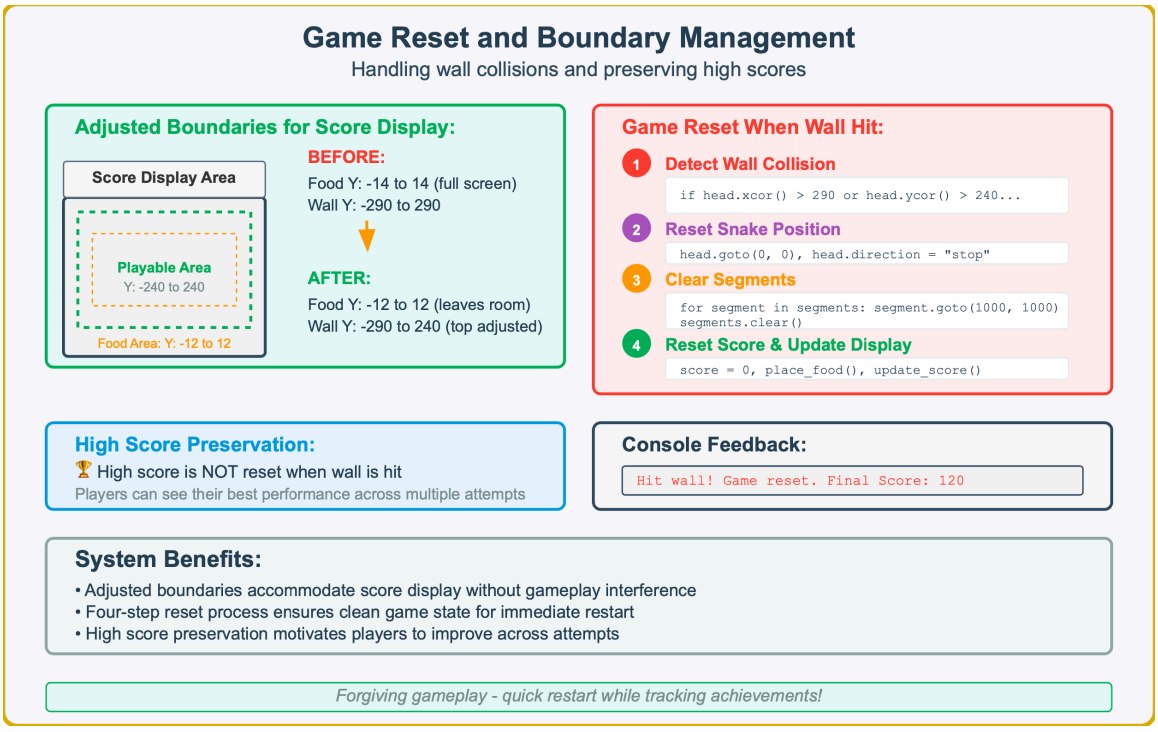

Figure 10.21 — Game Reset and Boundary Management System. The adjusted boundaries accommodate the score display area by changing food placement (Y: —12 to 12) and wall detection (Y: —290 to 240 for top wall). When wall collision occurs, the system follows a four—step reset: detect collision, reset snake position to center with direction “stop”, clear all segments, and reset score while preserving high score across attempts.

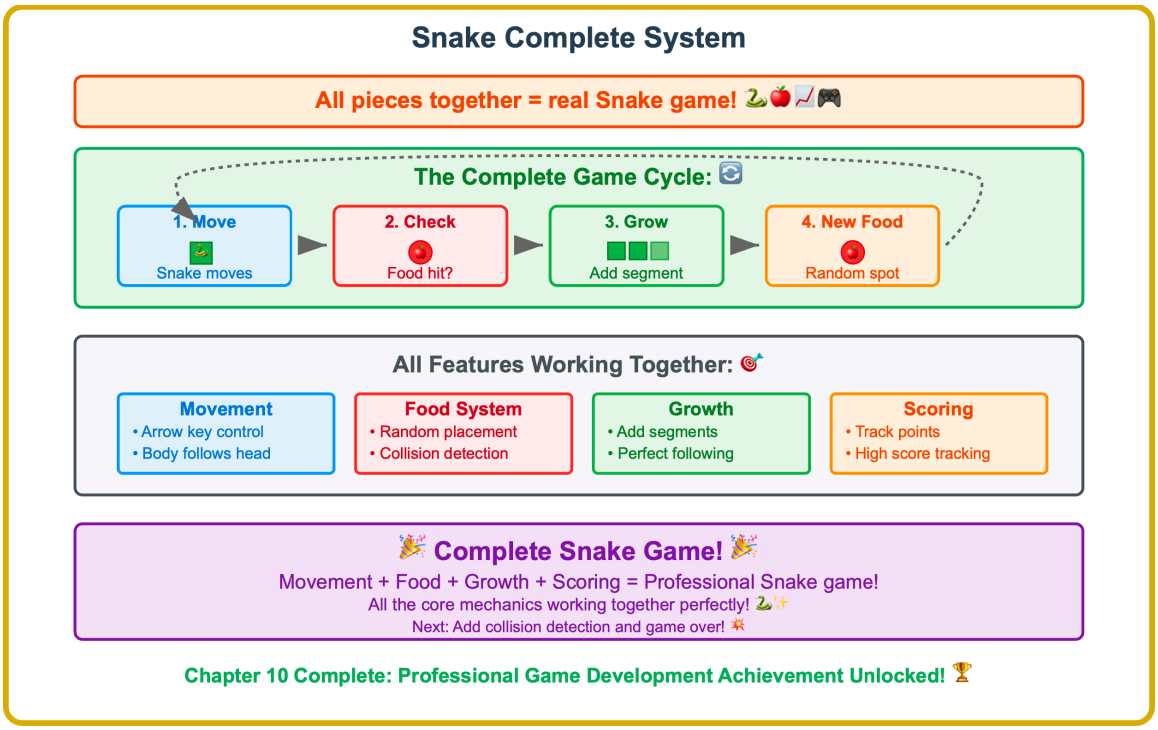

Figure 10.22 — Complete Snake Game System Integration. The complete game cycle shows all components working together: (1) Snake moves via arrow key control with body following head, (2) Food collision detection checks for hits, (3) Growth system adds segments with perfect following, (4) New food appears at random spots. All features integrate seamlessly: movement system, food system with random placement and collision detection, growth with segment addition, and scoring with point and length tracking.

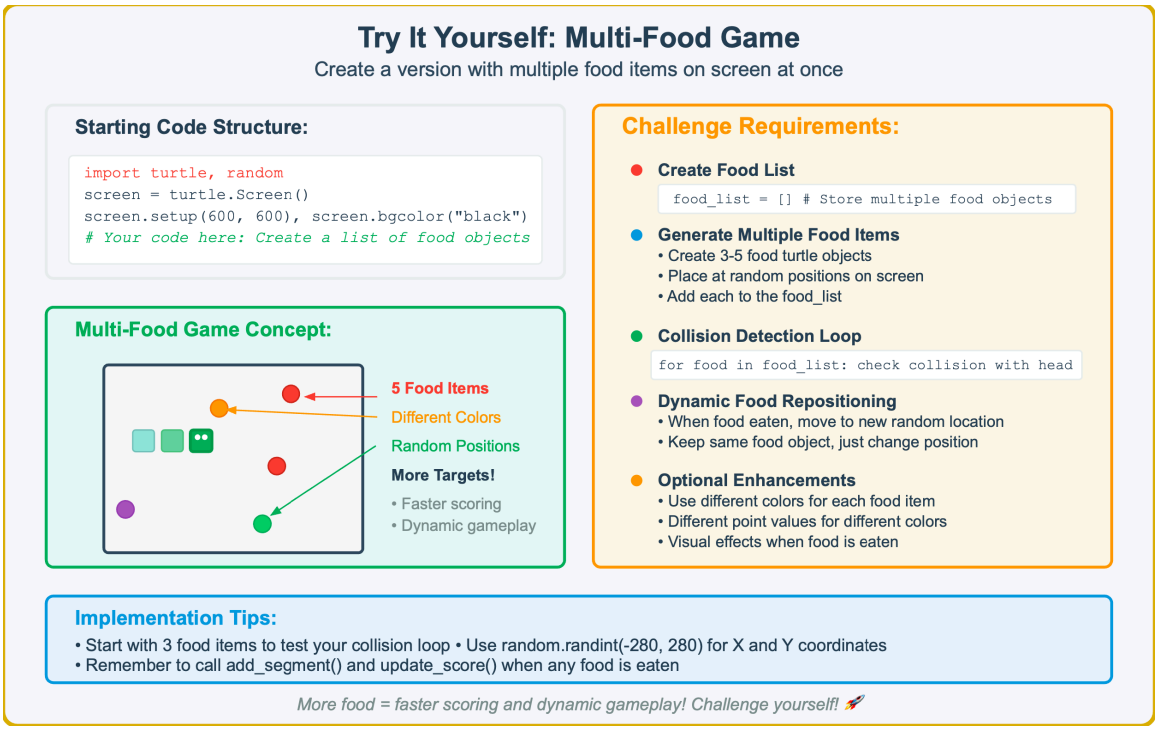

Figure 10.23 — Multi—Food Game Challenge — Extended Snake Mechanics. This programming challenge extends the basic Snake game by implementing multiple food items simultaneously on screen. Students create a list to store food turtle objects, generate 3—5 items at random positions with different colors, loop through all items for collision detection, and move eaten food to new random locations, creating faster scoring and more dynamic gameplay.

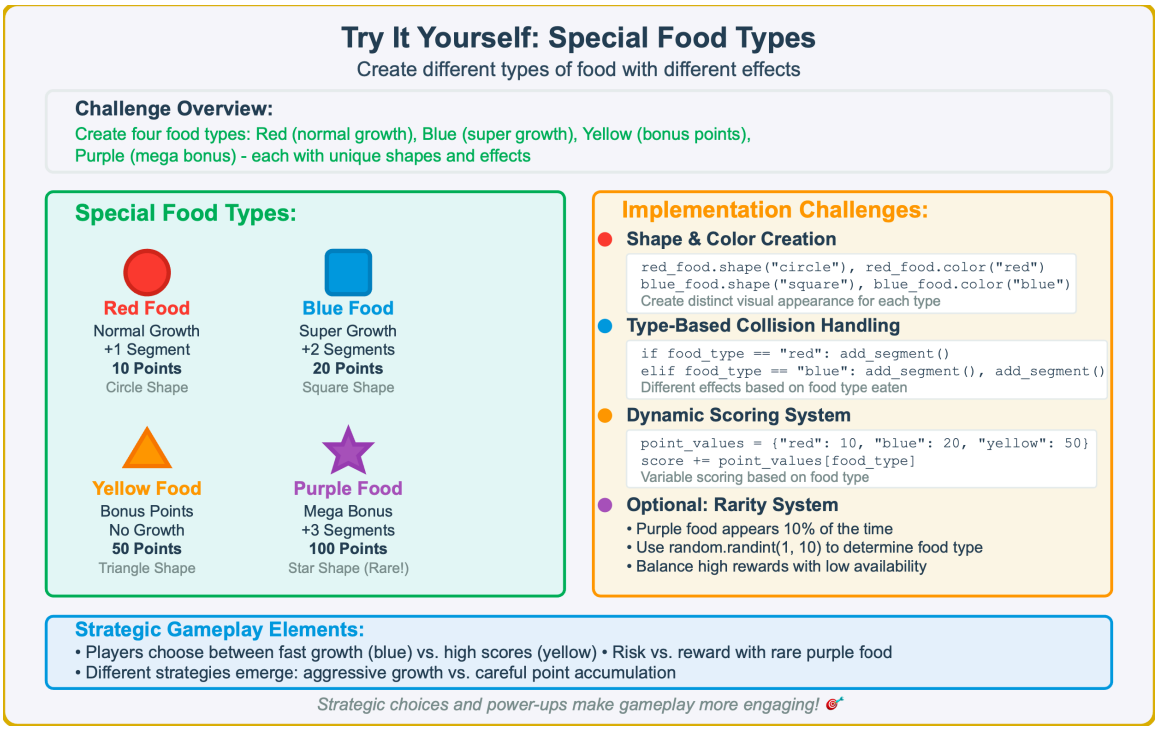

Figure 10.24 — Special Food Types Challenge — Strategic Gameplay Mechanics. This advanced challenge creates four distinct food types with different effects: Red food (circle, normal growth +1 segment, 10 points), Blue food (square, super growth +2 segments, 20 points), Yellow food (triangle, bonus points only, no growth, 50 points), and Purple food (star, mega bonus +3 segments, 100 points, rare appearance). Implementation requires different turtle shapes and colors, collision effect handling based on food type, and appropriate scoring/growth logic.

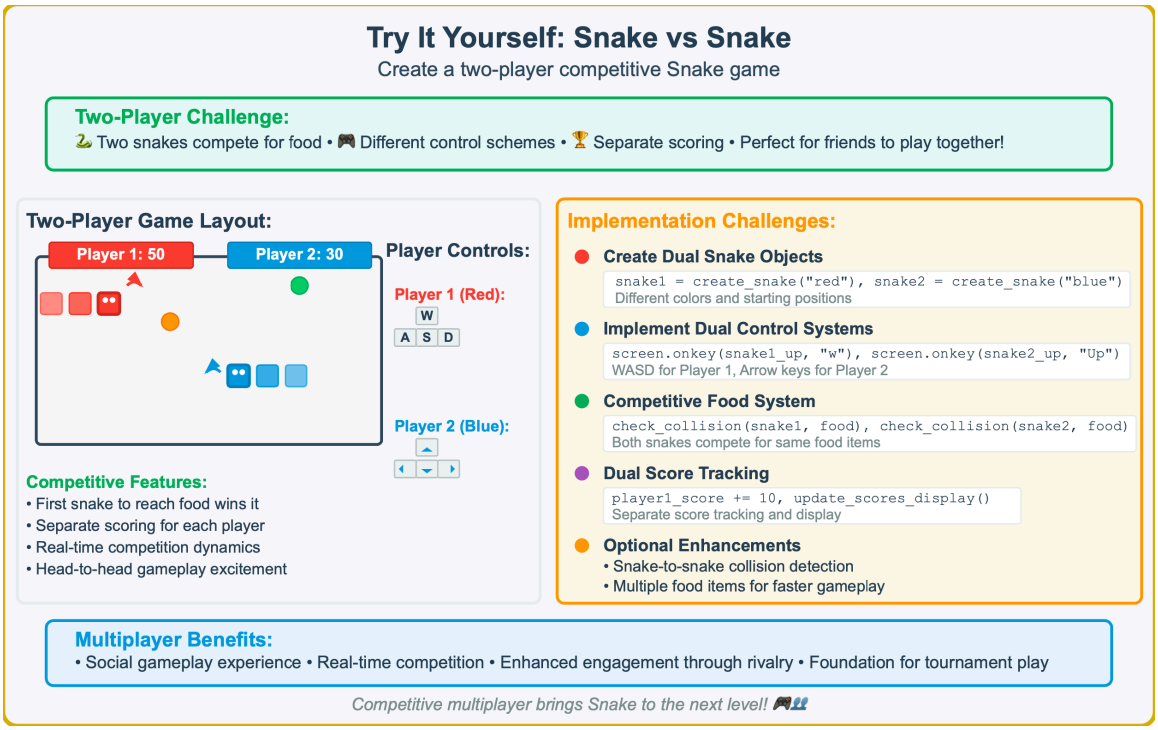

Figure 10.25 — Snake vs Snake Challenge — Two—Player Competitive Gaming. This multiplayer challenge creates a competitive two—player Snake game where red and blue snakes compete for food using different control schemes: Player 1 uses WASD keys (red snake), Player 2 uses arrow keys (blue snake). Implementation requires separate snake objects with different colors, dual control systems, shared food competition (first to reach wins), separate score tracking and display, with optional collision detection between snakes and multiple food items for enhanced gameplay.